11. Hafta: Rastgele Orman (Random Forest) Modeli

8.1. Random Forest modelinin uygulanması ve hiperparametrelerinin açıklanması

8.2. Modelin eğitilmesi

- **Özellik ölçekleme yöntemlerinin uygulanması**

8.3. Model sonuçlarının değerlendirilmesi

8.1. Random Forest modelinin uygulanması ve hiperparametrelerinin açıklanması

In [25]:
```
# Gerekli kütüphanelerin import edilmesi
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_validate, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from imblearn.under_sampling import RandomUnderSampler
from sklearn.pipeline import Pipeline
```

In [26]:
```
# Veri setini oku
df = pd.read_csv("cleaned_df_data.csv")
```

In [27]:
```
# burada kaç veri var
df.shape
```

Out[27]:
```
(45211, 15)
```

In [28]:
```
df
```

Out[28]:

	age	job	marital	education	default	housing	loan	contact	month	campaig
0	58	management	married	tertiary	0	1	0	unknown	may	
1	44	technician	single	secondary	0	1	0	unknown	may	
2	33	entrepreneur	married	secondary	0	1	1	unknown	may	
3	47	blue-collar	married	unknown	0	1	0	unknown	may	
4	33	unknown	single	unknown	0	0	0	unknown	may	
...	...	...	...	...	...	...	...	...	...	
45206	51	technician	married	tertiary	0	0	0	cellular	nov	
45207	71	retired	divorced	primary	0	0	0	cellular	nov	
45208	72	retired	married	secondary	0	0	0	cellular	nov	
45209	57	blue-collar	married	secondary	0	0	0	telephone	nov	
45210	37	entrepreneur	married	secondary	0	0	0	cellular	nov	

45211 rows × 15 columns

In [29]:
```
df.isna().any().sum()
```

Out[29]:
```
0
```

In [30]:
```
df
```

Out[30]:

	age	job	marital	education	default	housing	loan	contact	month	campaig
0	58	management	married	tertiary	0	1	0	unknown	may	
1	44	technician	single	secondary	0	1	0	unknown	may	
2	33	entrepreneur	married	secondary	0	1	1	unknown	may	
3	47	blue-collar	married	unknown	0	1	0	unknown	may	
4	33	unknown	single	unknown	0	0	0	unknown	may	
...	...	...	...	...	...	...	...	...	...	
45206	51	technician	married	tertiary	0	0	0	cellular	nov	
45207	71	retired	divorced	primary	0	0	0	cellular	nov	
45208	72	retired	married	secondary	0	0	0	cellular	nov	
45209	57	blue-collar	married	secondary	0	0	0	telephone	nov	
45210	37	entrepreneur	married	secondary	0	0	0	cellular	nov	

45211 rows × 15 columns

In [31]:
```
# y sınıf oranlarını kontrol et
print("Sınıf oranları:\n", df["y"].value_counts(normalize=True))
```

```
Sınıf oranları:
 0    0.883015
1    0.116985
Name: y, dtype: float64
```

Veri setimiz dengesiz bir yapıya sahip. Veri dengeleme yöntemlerinden.....kullanılacaktır.

8.2. Modelin eğitilmesi

- Özellik ölçekleme yöntemlerinin uygulanması

In [32]:
```
# Bağımlı ve bağımsız değişkenleri ayır
X = df.drop("y", axis=1)
y = df["y"]
```

In [33]:
```
# Kategorik değişkenleri dummies ile sayısallaştır
categorical_cols = X.select_dtypes(include="object").columns.tolist()
X_encoded = pd.get_dummies(X, columns=categorical_cols, drop_first=True)
```

In [34]:
```
X_encoded
```

Out[34]:

	age	default	housing	loan	campaign	previous	balance_level	has_any_credit	job_blue-collar
0	58	0	1	0	1	0	2	1	0
1	44	0	1	0	1	0	1	1	0
2	33	0	1	1	1	0	1	1	0
3	47	0	1	0	1	0	1	1	1
4	33	0	0	0	1	0	1	0	0
...	...	...	...	...	...	...	...	...	...
45206	51	0	0	0	3	0	1	0	0
45207	71	0	0	0	2	0	1	0	0
45208	72	0	0	0	5	3	2	0	0
45209	57	0	0	0	4	0	1	0	1
45210	37	0	0	0	2	11	2	0	0

45211 rows × 40 columns

In [35]:
```
# Eğitim-test ayrımı
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.3, ra
```

stratify=y parametresi, train-test ayrımında sınıf dengesini korumak için kullanılır.

In [36]:
```
# Ölçekleme
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [37]:
```
# Dengesiz veri problemi için RandomUnderSampler
rus = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = rus.fit_resample(X_train_scaled, y_train)
```

In [38]:
```
param_grid = {
    'n_estimators': [100],                  # 100 ağaç genellikle yeterlidir; çok
    'max_depth': [8, 10],                   # Daha düşük değerler daha az karmaşık
    'min_samples_split': [10],              # Daha fazla örnek gerektirerek aşırı
    'min_samples_leaf': [5, 10],            # Yaprakta en az bu kadar örnek olmalı
    'class_weight': ['balanced']            # Dengesiz sınıf problemini dengelemey
}
```

In [39]:
```
rf = RandomForestClassifier(random_state=42)
```

In [40]:
```
grid_search = GridSearchCV(
    rf,
    param_grid,
    scoring='f1',
    cv=5,
    n_jobs=-1,
    verbose=1
)
```

In [41]:
```
grid_search.fit(X_resampled, y_resampled)
best_model = grid_search.best_estimator_
```

```
Fitting 5 folds for each of 4 candidates, totalling 20 fits
```

In [42]:
```
print("\nEn iyi hiperparametreler:")
print(grid_search.best_params_)
```

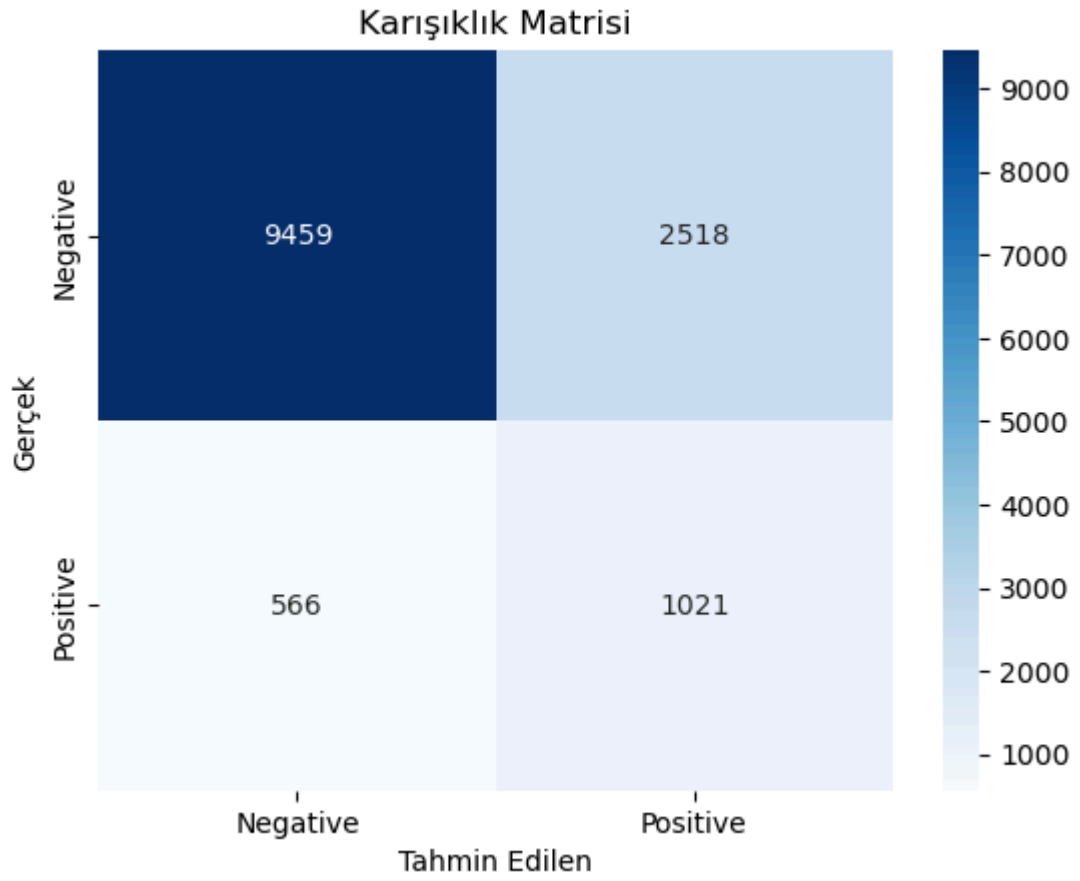
```
En iyi hiperparametreler:
{'class_weight': 'balanced', 'max_depth': 10, 'min_samples_leaf': 5, 'min_samples_
split': 10, 'n_estimators': 100}
```

8.3. Model sonuçlarının değerlendirilmesi

In [43]:
```
# TEST SETİ PERFORMANSI
y_test_pred = best_model.predict(X_test_scaled)

print("\nTEST SETİ PERFORMANSI:")
print(f"Accuracy : {accuracy_score(y_test, y_test_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_test_pred):.2f}")
print(f"Recall   : {recall_score(y_test, y_test_pred):.2f}")
print(f"F1 Score : {f1_score(y_test, y_test_pred):.2f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_test_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
```

```
TEST SETİ PERFORMANSI:
Accuracy : 0.77
Precision: 0.29
Recall   : 0.64
F1 Score : 0.40

Confusion Matrix:
[[9459 2518]
 [ 566 1021]]

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.79      0.86     11977
           1       0.29      0.64      0.40      1587

    accuracy                           0.77     13564
   macro avg       0.62      0.72      0.63     13564
weighted avg       0.87      0.77      0.81     13564
```

In [44]:
```python
# Karışıklık Matrisi (Heatmap)
cm = confusion_matrix(y_test, y_test_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Negative', 'Positive'],
            yticklabels=['Negative', 'Positive'])
plt.xlabel('Tahmin Edilen')
plt.ylabel('Gerçek')
plt.title('Karışıklık Matrisi')
plt.show()
```



In [45]:
```python
# EĞİTİM SETİ PERFORMANSI (Overfitting kontrolü)
y_train_pred = best_model.predict(X_resampled)
print("\nEĞİTİM SETİ PERFORMANSI:")
print(f"Accuracy : {accuracy_score(y_resampled, y_train_pred):.2f}")
```

```
print(f"Precision: {precision_score(y_resampled, y_train_pred):.2f}")
print(f"Recall   : {recall_score(y_resampled, y_train_pred):.2f}")
print(f"F1 Score : {f1_score(y_resampled, y_train_pred):.2f}")
```

```
EĞİTİM SETİ PERFORMANSI:
Accuracy : 0.74
Precision: 0.78
Recall   : 0.66
F1 Score : 0.71
```

- **Test setinde accuracy ve recall gayet iyi. Bu, modelin genel olarak iyi genelleştirdiğini gösteriyor.**
- **Precision testte düşük (0.29) → Yani model hâlâ bazı false positive'ler üretiyor, ama overfitting belirtisi değil; daha çok dengesiz sınıf dağılımının doğal sonucu olabilir.**
- **F1 skoru farkı fazla ama overfitting'in azaldığını gösteren diğer metriklerle birlikte bu kabul edilebilir.**

In [46]:
```
# Cross Validation (5-Fold)
print("\nCROSS VALIDATION (5-Fold) Sonuçları:")
cv_scores = cross_validate(
    best_model,
    X_resampled,
    y_resampled,
    cv=5,
    scoring=['accuracy', 'precision', 'recall', 'f1'],
    return_train_score=True
)

for metric in ['train_accuracy', 'test_accuracy', 'train_precision', 'test_precisio
               'train_recall', 'test_recall', 'train_f1', 'test_f1']:
    mean_score = cv_scores[metric].mean()
    print(f"{metric}: {mean_score:.2f}")
```

```
CROSS VALIDATION (5-Fold) Sonuçları:
train_accuracy: 0.74
test_accuracy: 0.71
train_precision: 0.79
test_precision: 0.75
train_recall: 0.67
test_recall: 0.63
train_f1: 0.72
test_f1: 0.69
```